ЛАБОРАТОРНА РОБОТА № 5

Тема: Робота з графічним інтерфейсом. Застосування компонентів Label, CheckBox, ComboBox, Entry, Button

Мета: Засвоїти принципи проектування та оголошення структур та вивчити особливості реалізації структур і їх методів.

Хід роботи

Завдання 1.

Оголосити структуру Product, яка представляє інформаціюпроодинтовар, який зберігається на складі. Має наступні поля: - Name – назва товару; - Price - вартість одиниці товару; - Cost – грошова одиниця, у якій вимірюється вартість; - Quantity – кількість наявних товарів на складі; - Producer – назва компанії-виробника; - Weight – вага одиниці товару. Оголосити структуру Currency, яка містить наступні поля: - Name – назва валюти; - ExRate – курс (дробове число - кількість гривень та копійок, що дають за одну одиницю валюти).

Код із файлу: main.go

Код програми:

```
type Currency struct {
    Name   string
    ExRate float64 // грн за 1 одиницю валюти
}

type Product struct {
    Name     string
    Price    float64  // в одиницях Cost
    Cost     Currency // грошова одиниця
    Quantity uint64
    Producer string
    Weight   float64 // вага одиниці товару
}
```

Результат виконання:

```
Товар #1
  Назва товару: сир
  Вартість одиниці (в валюті Cost): 20
  Назва валюти (Cost): гривні
  Курс валюти (грн за 1 Cost): 1:1
Некоректне число. Спробуйте ще раз.
  Курс валюти (грн за 1 Cost): 1
  Кількість на складі: 10
  Виробник: Наша Лінія
  Вага одиниці: 100г
Некоректне число. Спробуйте ще раз.
  Вага одиниці: 100
```

Рис. 5.1 - Лістинг коду

Завдання 2.

Для кожної структури реалізувати конструктори:

Виконання завдання:

Лістинг програми: Код з файлу - main.go

Код програми:

```
func NewCurrency(name string, exRate float64) Currency {
   return Currency{Name: name, ExRate: exRate}
}

func NewProduct(name string, price float64, cost Currency, quantity uint64, producer string, weight
float64) Product {
   return Product{
      Name:     name,
      Price:    price,
      Cost:     cost,
      Quantity: quantity,
      Producer: producer,
      Weight:   weight,
   }
}
```

Завдання 3.

Реалізувати set- та get- методи для кожного поля Product.

Код програми:

```
func (p *Product) SetName(name string)        { p.Name = name }
func (p Product) GetName() string             { return p.Name }
func (p *Product) SetPrice(price float64)   { p.Price = price }
func (p Product) GetPrice() float64         { return p.Price }
func (p *Product) SetCost(cost Currency)    { p.Cost = cost }
func (p Product) GetCost() Currency         { return p.Cost }
func (p *Product) SetQuantity(q uint64)     { p.Quantity = q }
func (p Product) GetQuantity() uint64       { return p.Quantity }
func (p *Product) SetProducer(s string)     { p.Producer = s }
func (p Product) GetProducer() string       { return p.Producer }
func (p *Product) SetWeight(w float64)      { p.Weight = w }
func (p Product) GetWeight() float64        { return p.Weight }
```

Завдання 4.

Код програми:

```
func (c Currency) ToUAH(amount float64) float64 {
return amount * c.ExRate
}
```

Завдання 5.

Код програми:

ReadProductsArray:

```
func ReadProductsArray() []Product {
   n := int(readUint("Введіть кількість товарів n: "))
   products := make([]Product, 0, n)
```

```
    for i := 0; i < n; i++ {
        fmt.Printf("\nТовар #%d\n", i+1)
        name := readLine("  Назва товару: ")
        price := readFloat("  Вартість одиниці (в валюті Cost): ", true, false)

        curName := readLine("  Назва валюти (Cost): ")
        exRate := readFloat("  Курс валюти (грн за 1 Cost): ", true, true) // курс > 0

        quantity := readUint("  Кількість на складі: ")
        producer := readLine("  Виробник: ")
        weight := readFloat("  Вага одиниці: ", true, false) // >= 0

        currency := NewCurrency(curName, exRate)
        products = append(products, NewProduct(name, price, currency, quantity,
producer, weight))
    }

    return products
}
```

PrintProduct:

```
func PrintProduct(p Product) {
    fmt.Println(p.String())
    fmt.Printf("  Загальна вага: %.2f\n", p.GetTotalWeight())
}
```

PrintProducts:

```
func PrintProducts(products []Product) {
    fmt.Println("\nСписок товарів:")
    for _, p := range products {
        PrintProduct(p)
        fmt.Println()
```

```
    }
}
```

GetProductsInfo:

```
func GetProductsInfo(products []Product, cheapest *Product, mostExpensive *Product) {
    if len(products) == 0 || cheapest == nil || mostExpensive == nil {
        return
    }

    min := products[0]
    max := products[0]

    for i := 1; i < len(products); i++ {
        if products[i].GetTotalPrice() < min.GetTotalPrice() {
            min = products[i]
        }
        if products[i].GetTotalPrice() > max.GetTotalPrice() {
            max = products[i]
        }
    }

    *cheapest = min
    *mostExpensive = max
}
```

Висновок

У результаті виконання лабораторної роботи №5 було закріплено практичні навички об’єктно-орієнтованого підходу в мові Go через використання структур,

методів та елементів інтерфейсного стилю програмування. Реалізовано структури Product і Currency, для них створено конструктори, а також set/get-методи для керування полями об’єктів. Розроблено методи обчислення ціни товару в гривнях, загальної вартості партії та сумарної ваги, що дало змогу виконувати прикладні розрахунки для складського обліку.

Крім того, реалізовано функції зчитування масиву товарів з клавіатури, виведення даних та визначення найдешевшого і найдорожчого товару. У програмі передбачено перевірку коректності введення, що підвищує надійність роботи. Отже, поставлену мету лабораторної роботи досягнуто повністю: засвоєно принципи проєктування структур і методів у Go та отримано практичний досвід створення завершеної програми для обробки структурованих даних.

Контрольні питання

1. Що таке структура і для чого призначена?
Структура — це користувацький тип, який об'єднує декілька полів різних типів в одну сутність. Вона використовується для представлення об'єктів предметної області.

2. Як створюються користувацькі типи?
Через ключове слово `type`, наприклад: `type Product struct { ... }`.

3. Що таке ООП?
ООП — це підхід до програмування, у якому дані та пов'язані з ними дії об'єднуються у типи/об'єкти. Основні принципи: інкапсуляція, поліморфізм, успадкування (у Go замість класичного успадкування переважає композиція).

4. Для чого використовується вбудова користувацьких типів?
Для композиції та повторного використання коду: вбудований тип передає свої поля і методи зовнішньому типу.

5. Що таке інтерфейси і для чого вони призначені?
Інтерфейс — це набір сигнатур методів. Він дозволяє працювати з різними типами однаково, якщо вони реалізують потрібні методи.

Файли виконаної роботи
- main.go — програмна реалізація завдань лабораторної №5;
- go.mod — файл модуля Go;
- README.md — коротка інструкція запуску.